Instructions système

Pour voir un exemple d'instructions système, exécutez le notebook "Intro to Gemini 2.0 Flash" dans l'un des environnements suivants :

Ouvrir dans Colab (https://colab.research.google.com/github/GoogleCloudPlatform/generative-ai/blob/main/gemini/getting-started/intro_gemini_2_0_flash.ipynb?hl=fr)

|

Ouvrir dans Colab Enterprise (https://console.cloud.google.com/vertex-ai/colab/import/https%3A%2F%2Fraw.githubusercontent.com%2FGoogleCloudPlatform%2Fgenerative-ai%2Fmain%2Fgemini%2Fgetting-started%2Fintro_gemini_2_0_flash.ipynb?hl=fr)

|

Ouvrir dans Vertex AI Workbench (https://console.cloud.google.com/vertex-ai/workbench/deploy-notebook?download_url=https%3A%2F%2Fraw.githubusercontent.com%2FGoogleCloudPlatform%2Fgenerative-ai%2Fmain%2Fgemini%2Fgetting-started%2Fintro_gemini_2_0_flash.ipynb&hl=fr)

|

Afficher sur GitHub (https://github.com/GoogleCloudPlatform/generative-ai/blob/main/gemini/getting-started/intro_gemini_2_0_flash.ipynb)

Ce document décrit les instructions système et les bonnes pratiques à suivre pour écrire des instructions système efficaces. Pour savoir comment ajouter des instructions système à vos requêtes, consultez plutôt la section Utiliser des instructions système (https://docs.cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/system-instructions?hl=fr).

Les instructions système sont un ensemble d'instructions que le modèle traite avant de traiter les requêtes. Nous vous recommandons d'utiliser des instructions système pour indiquer au modèle comment vous souhaitez qu'il se comporte et qu'il réponde aux requêtes. Par exemple, vous pouvez inclure des éléments tels qu'une persona à adopter, des informations contextuelles et des instructions de mise en forme.

Les instructions système sont particulièrement utiles lorsque vous souhaitez fournir au modèle des informations qu'un utilisateur final ne peut pas voir ni modifier. Cela peut vous aider à fournir au modèle un contexte supplémentaire pour qu'il comprenne la tâche, à

fournir des réponses plus personnalisées et à respecter des consignes spécifiques pendant toute l'interaction de l'utilisateur avec le modèle. De plus, les instructions système sont utiles lorsque vous souhaitez exécuter plusieurs requêtes avec le même ensemble d'instructions.

Lorsque des instructions système sont définies, elles s'appliquent à l'ensemble de la requête. Les instructions système fonctionnent sur plusieurs tours d'utilisateur et de modèle lorsqu'elles sont incluses dans la requête. Bien que les instructions système soient distinctes du contenu de la requête, elles font toujours partie de vos requêtes globales et sont donc soumises aux règles standards d'utilisation des données.

Remarque : Les instructions système peuvent aider le modèle à suivre les instructions, mais elles n'empêchent pas complètement les jailbreaks ou les fuites. Nous vous recommandons de faire preuve de prudence lorsque vous insérez des informations sensibles dans les instructions système.

Cas d'utilisation

En règle générale, nous vous recommandons d'utiliser les instructions système pour effectuer une ou plusieurs des opérations suivantes :

- Définir un persona ou un rôle
- Définir le format de sortie
- Définir le style et le ton de la sortie
- Définir des objectifs ou des règles pour la tâche
- Fournir du contexte supplémentaire pour la requête

Persona ou rôle ... Format de sortie ... Style et ton (#style-et-ton) Objectifs et (#objecti

Vous pouvez influencer la réponse d'un modèle en définissant des objectifs ou des règles, c'est-à-dire ce que le modèle doit ou ne doit pas faire, dans les instructions système.

Par exemple, supposons que vous souhaitiez utiliser un modèle pour écrire du code Python. Comparez les réponses du modèle lorsque vous définissez les règles suivantes :

- Lorsque vous générez du code, veillez à inclure des docstrings expliquant les entrées, les sorties et l'utilisation de chaque méthode.
- Générez uniquement du code, n'incluez pas de docstrings.

	Docstrings	Aucune docstring
Instructions système	Lorsque vous générez du code, veillez à inclure des docstrings expliquant les entrées, les sorties et l'utilisation de chaque méthode.	Générez uniquement du code, n'incluez pas de docstring.
Requête	Créez un tableau 2D en Python et un algorithme de recherche de chemin A* pour identifier le chemin le plus court entre deux points de la grille.	
Réponse		

```
import heapq

class Node:
    """"""
    Represents a node in the gr

    Attributes:
        x (int): The x-coordina
        y (int): The y-coordina
        parent (Node, optional)
        g (int): Cost from the
        h (int): Estimated cost
    """"""

    def __init__(self, x, y, pa
        self.x = x
        self.y = y
        self.parent = parent
        self.g = 0
        self.h = 0

    def __lt__(self, other):
        """"""Compares nodes ba
        return (self.g + self.h

def a_star(grid, start, end):
    """"""
    Implements the A* pathfindi

    Args:
        grid (list): A 2D array
        start (tuple): The star
```

```
import heapq

class Node:
    def __init__(self, position
        self.position = positic
        self.parent = parent
        self.g = 0
        self.h = 0
        self.f = 0

    def __lt__(self, other):
        return self.f < other.f

def astar(grid, start, end):
    rows = len(grid)
    cols = len(grid[0])

    open_list = []
    closed_list = set()

    start_node = Node(start)
    end_node = Node(end)

    open_list.append(start_node

    while open_list:
        current_node = heapq.he

        if current_node.positic
            path = []
            while current_node:
                path.append(cur
```

```
    end (tuple): The end pc

Returns:
    list: A list of coordir
"""""""

rows = len(grid)
cols = len(grid[0])

# Create start and end node
start_node = Node(start[0],
end_node = Node(end[0], enc

# Initialize open and close
open_list = []
closed_list = set()

# Add the start node to the
heapq.heappush(open_list, s

while open_list:
    # Get the node with the
    current_node = heapq.he

    # Add the current node
    closed_list.add((currer

    # Check if we've reache
    if current_node.x == er
        # Reconstruct and r
        path = []
        while current_node:
            path.append((cu
            current_node =
        return path[::-1]

    # Explore neighboring r
    for dx, dy in [(0, -1),
        neighbor_x = currer
        neighbor_y = currer

        # Check if the neig
        if 0 <= neighbor_x
            # Check if the
            if grid[neighbc
                # Create a
                neighbor_nc
                current_node =
            return path[::-1]

        closed_list.add(current

        for neighbor in get_nei
            if neighbor.positic
                continue

            if neighbor not in
                neighbor.g = cu
                neighbor.h = ma
                neighbor.f = ne
                neighbor.parent
                heapq.heappush(
            else:
                if neighbor.g >
                    neighbor.g
                    neighbor.f
                    neighbor.pa
                    heapq.heapi

    return None  # No path four

def get_neighbors(grid, node):
    rows = len(grid)
    cols = len(grid[0])
    row, col = node.position
    neighbors = []
    if row > 0 and grid[row - 1
        neighbors.append(Node((
    if row < rows - 1 and grid[
        neighbors.append(Node((
    if col > 0 and grid[row][cc
        neighbors.append(Node((
    if col < cols - 1 and grid[
        neighbors.append(Node((
    return neighbors

def manhattan_distance(a, b):
    return abs(a[0] - b[0]) + a

# Example usage
grid = [
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
```

```
                        # Calculate        [0, 0, 0, 0, 0],
                        neighbor_nc    ]

                        # Calculate    start = (0, 0)
                        neighbor_nc    end = (4, 4)

                        # Add the r    path = astar(grid, start, end)
                        if neighbor    print(path)
                            heapq.h

        # No path found
        return None
```

(gemini-pro)

```
    # Example usage:
    grid = [
        [0, 0, 0, 0, 0],
        [0, 1, 0, 1, 0],
        [0, 0, 0, 0, 0],
        [0, 1, 1, 1, 0],
        [0, 0, 0, 0, 0]
    ]

    start = (0, 0)
    end = (4, 4)

    path = a_star(grid, start, end)

    if path:
        print(""Shortest path:"", p
    else:
        print(""No path found."")
```

Explication :

1. Classe de nœud :
   - Représente une cellule de la grille.
   - Stocke les coordonnées (**x**, **y**), le nœud parent, **g** (coût à partir du début) et **h** (heuristique à la fin).
   - `__lt__` permet de comparer les nœuds de la file d'attente prioritaire (à l'aide de **heapq**) en fonction de **f = g + h**.
2. Fonction a_star :

- Utilise les points `grid`, `start` et `end` comme entrée.
- Initialise `open_list` (nœuds à explorer) et `closed_list` (nœuds explorés).
- Utilise `heapq` pour récupérer efficacement le nœud avec le score `f` le plus bas de `open_list`.
- Effectue des boucles jusqu'à ce que `end` soit trouvé ou que `open_list` soit vide (aucun chemin).
- Explore les voisins, calcule les coûts (`g`, `h`) et met à jour `open_list`.
- Reconstitue et renvoie le chemin d'accès lorsque `end` est atteint.

3. Exemple d'utilisation :
   - Crée un exemple de `grid` et définit les points `start` et `end`.
   - Appel de `a_star` pour trouver le chemin.
   - Imprime le chemin le plus court ou indique si aucun chemin n'existe.

(gemini-pro)

Étape suivante

- Découvrez comment utiliser les instructions système (https://docs.cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/system-instructions?hl=fr)
- Découvrez d'autres exemples de requêtes dans la galerie de requêtes (https://docs.cloud.google.com/vertex-ai/generative-ai/docs/prompt-gallery?hl=fr).

Sauf indication contraire, le contenu de cette page est régi par une licence Creative Commons Attribution 4.0 (https://creativecommons.org/licenses/by/4.0/), et les échantillons de code sont régis par une licence Apache 2.0 (https://www.apache.org/licenses/LICENSE-2.0). Pour en savoir plus, consultez les Règles du site Google Developers (https://developers.google.com/site-policies?hl=fr). Java est une marque déposée d'Oracle et/ou de ses sociétés affiliées.

Dernière mise à jour le 2026/02/27 (UTC).